

School of Computer Science, McGill University

COMP-421 Database Systems, Winter 2025*

Written Assignment Description 4: Query Evaluation, Mapy-Reduce and (Transaction Management)

Due Date April-11, 12:00 NOON

There are a total of 70 points on this assignment!

Note that some questions below have zero points. You do not need to submit them for the assignment but they might be useful for general practice and preparation for the final exam.

Provide your solution in a single file. This is an individual assignment.

Part I: Query Evaluation

We continue with the same database as for the last assignment. Below information is nearly the same as for A3 but it has some additional information that is needed to answer the questions in this assignment. .

In this assignment, we evaluate queries for a banking - credit card database. We look at three relations **Account**, **Card**, and **Payment**, listed below.

Account (accountid:INT, name:VARCHAR(30), email:VARCHAR(50), address:VARCHAR(60), balance:INT, accounttype:VARCHAR(12))
9012101, 'Joe Rich', joe.rich@maildomain.com '1180 Rue. St. Mathieu, Montreal', 2500.0, 'checking'

Card (cardid:INT, owneraccountid:INT, paymentdue:INT, cardtype:CHAR(7))
→ owneraccountid references Account.
122202, 9012101, 250.0, 'AIRPLAN'

Payment (paymentid:INT, srcaccountid:INT, cardid:INT amount:INT, paydate:CHAR(10))
→ srcaccountid references Account, cardid references Card.
6223451, 9012101, 122202, 205.0, '2025-03-05'

INT has 4 Bytes, a char has 1 Byte. Assume for each VARCHAR attribute in the Account table, the average size of a value is half the total capacity allocated for that attribute. (e.g: **name** is on an average 15 characters).

There are 8,000 members in the bank, each with one account (no joint accounts, etc.) and 10,000 records in the **Card** relation . Half of the members have exactly two cards under their account. A quarter of the members have exactly one card. The rest have no cards. As such, there are 10,000 records in the **Card** relation.

*Copyright by Bettina Kemme; All material provided to the students as part of this course is the property of respective authors. Publishing it to third-party (including websites) is prohibited. Students may save it for their personal use, indefinitely, including personal cloud storage spaces. Further, no assessments published as part of this course may be shared with anyone else.

Payments are made on an average once per month per card. The database has 1 full year of data stored in it (12 months - 365 days). As such, there are 120,000 payments stored in the **Payments** relation. A payment can be made from an account that is not necessarily the account attached to the card. (ie., **srcaccountid** associated with a particular payment of a card need not be the **owneraccountid** associated with the card itself. Instead, assume it's randomly distributed across all accounts. As in, someone else paid off the bill).

Account balance and **Payment amount** can each take on 3,000 different values, uniformly distributed between 1 and 3000. For simplicity, we only consider integer values for payments (that is, no floats or decimal data types).

cardtype takes on 3 different values and **accounttype** takes on 5 different values, both uniformly distributed.

Payments are equally likely to be made on any of the 365 days of the year.

In general all database pages are 4,000 bytes with an average 75% fill factor. The only exception is if a single index data entry will take more than 75% of a page, then the database will fill as much space in that page as is required to keep that data entry completely in that leaf page. Also the root might have any fill factor.

Rids are 10 bytes each and internal page pointers are 6 bytes each. A data entry is NOT split across multiple pages.

You can assume that the root and intermediate nodes of an index are always in the memory (so you need not account for them in your memory requirements).

You can also assume there are 50 free buffer pages available in any question where this information is required, but you can add a few extra ones for the sake of simplifying calculations.

For simplicity, you can assume that the **Account** relation is split across 250 pages, the **Card** relation has 50 pages, and **Payment** has 1000 pages. (Feel free to calculate the actual estimated page numbers.)

Furthermore, you can assume an unclustered, type II B+-tree for **Payment** on **paydate** attribute. This index has 365 index leaf pages (in fact, each data entry is on a different leaf page)

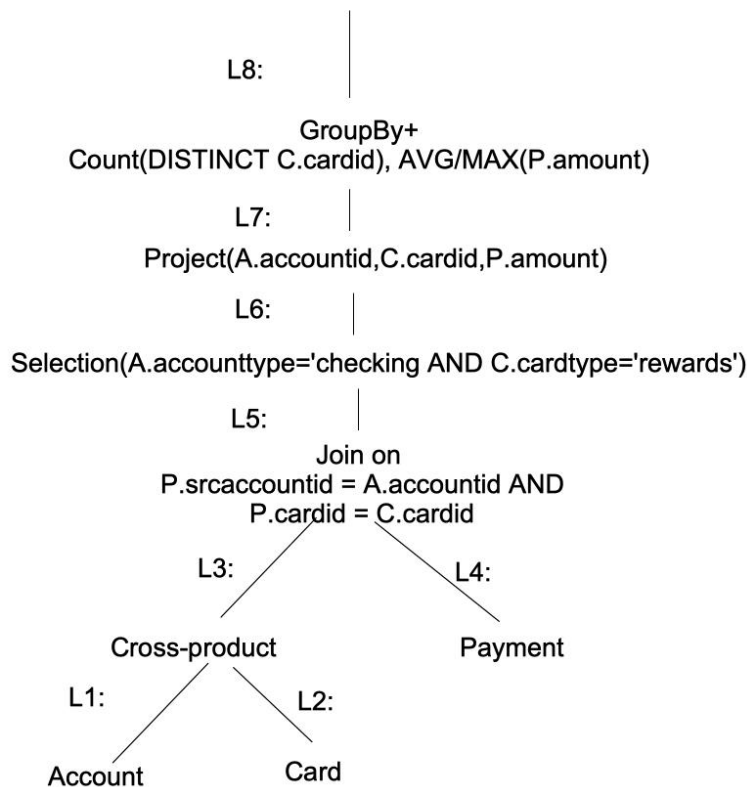
Ex. 1 — Cardinality Estimation and Algebraic Query Optimization (22 points)

Consider the following query:

```
SELECT A.accountid, COUNT(DISTINCT C.cardid), AVG(P.amount), MAX(P.amount)
FROM Account A, Card C, Payment P
WHERE P.srcaccountid = A.accountid AND
      P.cardid = C.cardid AND
      A.accounttype='checking' AND
      C.cardtype='rewards'
GROUP BY A.accountid
```

The execution tree below is a non-optimized algebraic expression of this query. Note that for brevity, we refer to the tables **Account** as **A**, **Card** as **C** and **Payment** as **P**:

- 1.(12 points) *Indicate the number of tuples and the size of the tuples that flow from the base relations to the first operator and then from one operator to the next. These positions are indicated with L1 to L8.*
- 2.(10 points) *Now draw the execution plan after performing an algebraic optimization (pushing down selections, pushing down projections as far as possible, and avoiding cross-product whenever possible. Note that you do not need to indicate how each of the operations would be executed.*
- 3.(0 points) *Indicate the number of tuples that flow from the base relations to the first operator and then from one operator to the next for your optimized execution plan.*



Ex. 2 — Cost-based Query Optimization (20 Points)

Consider the query below. For each card type, it counts the number of associated accounts with balance less than 6.

```

SELECT C.cardtype, COUNT(DISTINCT C.owneraccountid)
FROM Account A, Card C
WHERE A.accountid = C.owneraccountid
AND A.balance < 6.0
GROUP BY C.cardtype

```

Find the optimal execution plan and its estimated I/O cost. Make sure to describe how each of the operators is executed (scan, index, type of join, on-the-fly, requires I/O write for intermediate

results or buffered, etc.). Provide reasoning why this is a good plan (you could, e.g., provide the calculation of alternative plans and with this show that the one you present is the fastest one).

Note that you are not required to draw the algebraic optimisation tree, but that doing so might help you find a solution more quickly.

Assume that **Account** has an index on **accountid** and **Card** has an index on **owneraccountid**. Both indexes are unclustered type II.

Ex. 3 — Map Reduce (28 points)

Now assume that above relations are much larger. As such, we want to use a map-reduce framework to answer certain queries.

Write workflows with map/reduce functions for the following tasks. Note that it is possible that to solve the query, you need a sequence of map/reduce pairs, where the output of the first reduce is input to the second map.

- 1.(8 points) *Output the account ids of those accounts that have two cards. Make sure that you output every account id only once.*
- 2.(10 points) *Indicate how many accounts have 1, 2, 3, etc. cards. The description above indicates that an account has 0, 1 or 2 cards. But create a program here that can handle an arbitrary number of cards. Thus, the output should have tuples (1, x1), (2, x2), (3, x3), etc. where x_i is the number of accounts that have i cards. If there is no account that has i cards, then you do not need to output a record for i .*
- 3.(10 points) *Adjust the previous query to also output how many accounts have 0 cards.*

Use the following format to present map/reduce

Map:

```
For each input record rm of relation X
do something
output ...
```

Reduce

```
For each input record rr
do something
output ...
```

Where **rm** has the format (key, value) (value can be split into several attributes) and **rr** has the format (key, list-of-values) (each value can be split into list of attributes).

Ex. 4 — Schedules and Concurrency Control (0 Points)

Given the following three schedules S1, S2, S3.

S1			S2			S3		
T1	T2	T3	T1	T2	T3	T1	T2	T3
w1(c)	r2(b)	r3(a)	r1(a)		r3(c)	r1(a)	w2(a)	r3(a) w3(b) c3
	w2(b)		w1(a)	w2(b)				
	r2(a) c2	r3(c)		r2(c) w2(c) c2		w1(a)	r2(b)	
r1(b)			w1(c)				w2(c)	
w1(b)			c1				c2	
c1		w3(b) c3			r3(a) c3	c1		

- For each of the schedules, show the serialization graph. Indicate whether the schedule is serializable or not. If it is serializable, provide an equivalent serial schedule.
- For each of the schedules, indicate whether it has (a) a dirty read, (b) lost update, (c) unrepeatable read
- Assume now that the figures above does not depict the final schedules but the sequences of operations submitted to the system. Now assume a DBMS that uses strict 2PL for concurrency control. Describe how strict 2PL handles each of the sequences above. Add lock $S_i(a)$ when T_i acquires shared lock on a , $X_i(a)$ when T_i acquires exclusive lock on object a , and unlock request $U_i(a)$ when T_i releases any lock on object a to the above sequence. The DBMS processes actions in the order shown. If a transaction is blocked, assume that all of its actions are queued until it is resumed; the DBMS continues with the next action (according to the listed sequence) of an unblocked transaction.

Distinguish two variations of the locking scheme considering getting a shared lock when only shared locks are active. (1) Upon a shared lock request $S_i(a)$ of transaction T_i if there are only shared locks active on a , then grant $S_i(a)$, else $S_i(a)$ must wait (i.e., do not consider locks waiting); (2) Upon a shared lock request $S_i(a)$ of transaction T_i , if there are only shared locks active on a and no lock is waiting on a , then grant $S_i(a)$, else $S_i(a)$ must wait.

Furthermore, upon a shared lock request $X_i(x)$ of transaction T_i if the only lock active on x is a shared lock $S_i(x)$ of the same transaction, then grant $X_i(x)$ (even if other locks are waiting).

Ex. 5 — Schedules and SQL (0 points)

Assume relation `Enrolled(sid:INT, cid:CHAR(10), grade:INT)`

with example instance:

sid	cid	grade
4711	COMP-421	92
4711	MATH-240	82

Assume a database that does not have any concurrency control in place. For each of the executions below (starting with the example instance above), indicate

- 1.the values returned by the SELECT statements and the final value of the database.
- 2.whether the execution violates the anomalies (i) dirty read, (ii) lost update, (iii) unrepeatable read

(NOTE that the execution could violate all anomalies, or only some or none could hold).

(I)

```
T1: SELECT grade FROM Enrolled WHERE sid = 4711
T2: UPDATE Enrolled SET grade = grade + 5 WHERE cid = 'COMP-421'
T1: UPDATE Enrolled SET grade = 84 WHERE sid = 4711 AND cid = 'COMP-421'
T1: COMMIT
T2: COMMIT
```

(II)

```
T1: SELECT sid FROM Enrolled WHERE grade > 90
T2: UPDATE Enrolled SET grade = grade + 10 WHERE grade < 85 AND cid = 'COMP-421'
T1: SELECT sid FROM Enrolled WHERE grade > 80 AND grade <= 90
T1: COMMIT
T2: COMMIT
```

(III)

```
T1: UPDATE Enrolled SET grade = grade + 5 WHERE cid = 'MATH-240'
T2: UPDATE Enrolled SET grade = grade + 5 WHERE cid = 'COMP-421'
T3: SELECT cid, AVG(grade) FROM Enrolled GROUP BY cid
T2: ABORT
T3: ABORT
T1: COMMIT
```

(IV)

```
T1: SELECT cid, avg(GRADE) FROM Enrolled GROUP BY cid
T2: UPDATE Enrolled SET grade = grade + 5 WHERE cid = 'COMP-421'
T2: COMMIT
T1: SELECT sid, avg(GRADE) FROM Enrolled GROUP BY sid
T1: COMMIT
```

(V)

```
T1: SELECT * FROM Enrolled WHERE sid = 4711
T2: SELECT * FROM Enrolled WHERE sid = 4711
```

```
T1: UPDATE Enrolled SET grade = 100 WHERE sid = 4711 AND cid = 'COMP-421'
T2: UPDATE Enrolled SET grade = 100 WHERE sid = 4711 and cid = 'MATH-240'
T1: COMMIT
T2: COMMIT
```

Hints & Guidelines Query Evaluation

Note that these guidelines have been developed for query evaluation questions over the years. Specific points might not be relevant for the particular assignment of a given year.

- You do not have to “budget” for slot directory, sibling pointers, row header, etc., when doing your page size calculations - only data entries, index entries and record sizes need to be considered.
- Assume that a NULL value takes no extra space in the data record (table, intermediate results, etc.).
- To simplify the computation of index logistics, you can assume that the size of NULL values stored in the index is the same as the size of regular values of that column.
- Assume uniform distribution of values, unless the information given indicates otherwise.
- You can use the information (leaf pages, data entries, etc.) computed in one Exercise, in the following exercises.
- Use pipelining when possible to save I/O cost between multiple steps (operators) in the execution plan. Not doing this could result in points lost (depending on how straightforward it would have been to integrate).
- When calculating I/O, take into account the I/O cost savings due to pipelining in your equations.
- You can leave fractions as it is, if you would like to (as in number of data entries per page, records per page, etc.). However, keep in mind that there are some steps that cannot produce fractions. For example, if you have a specific block nested join step in your execution plan, its cost cannot be 24.6 pages, it has to be 27 pages - because that is the unit in which we do I/O (same with memory buffers).
- Minor rounding errors due to approximating data entries per page, records per page, etc., are acceptable. In general look at the magnitudes of the numbers you are rounding and how big a number you are multiplying it with later to see how significant the impact is. (for example, rounding 1.5 to 2 and rounding 1000.5 to 1001 introduces very different magnitude shifts to the numbers with which they are multiplied later). At the end of the day, remember that our objective is to have a reasonably good computational methodology to help us choose between multiple possible execution plans based on their costs.

- When making execution plans, remember that any temporary intermediate pages, etc., that you create as part of query execution can be packed near 100% to maximally use the memory/disk and reduce I/O cost. Do not leave them at the fill factor of regular database pages (of tables and indexes) that are usually below 100%.
- Partial points are given to steps and methodology even if you making errors doing math. But try to use a calculator to do your math to avoid making mistakes.